

SZAKDOLGOZAT



MISKOLCI EGYETEM

A C# programozás digitális oktatásának fejlesztése játékos és interaktív módszerekkel

Készítette:

Balogh Martin

Programtervező informatikus

Témavezető:

Dr. Baksáné Dr. Varga Erika (egyetemi docens)

MISKOLC, 2026

MISKOLCI EGYETEM

Gépészmérnöki és Informatikai Kar
Alkalmazott Matematikai Intézeti Tanszék

Szám:

SZAKDOLGOZAT FELADAT

Balogh Martin (OIKGCW) programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: C# oktatás

A szakdolgozat címe: A C# programozás digitális oktatásának fejlesztése játékos és interaktív módszerekkel

A feladat részletezése:

A dolgozat célja egy meglévő Moodle tananyag bővítése a C# programozás oktatásának hatékonyabb támogatására. A hallgató a tananyagot kiegészíti egy Unity engine-ben, C# nyelven fejlesztett interaktív oktató játékkal, amely játékos formában segíti a programozási alapismeretek gyakorlását. A fejlesztett rendszer tartalmaz egy egyszerű AI funkciót, ami neurális háló alapú visszacsatolást nyújt a játékbeli teljesítmény alapján.

A megoldandó feladatok:

- A meglévő Moodle tananyag áttekintése és bővítési lehetőségeinek azonosítása.
- Oktatási célú játék tervezése és fejlesztése Unity engine-ben, C# nyelven.
- Egyszerű, neurális háló alapú visszacsatolási logika tervezése és implementálása.
- Az elkészült oktatási segédeszköz integrálása a Moodle keretrendszerbe.

Témavezető: Dr. Baksáné Dr. Varga Erika Egyetemi docens

A feladat kiadásának ideje: 2025. szeptember 20.

.....
szakfelelős

EREDETISÉGI NYILATKOZAT

Alulírott **Balogh Martin**; Neptun-kód: OIKGCW a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírással igazolom, hogy *A C# programozás digitális oktatásának fejlesztése játékos és interaktív módszerekkel* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, év hó nap

.....
Hallgató

- | | | |
|----|-----------------------------------|--|
| 1. | A szakdolgozat feladat módosítása | szükséges (módosítás külön lapon)
nem szükséges |
|----|-----------------------------------|--|

.....
dátum

.....
témavezető(k)

2. A feladat kidolgozását ellenőriztem:

témavezető (dátum, aláírás):

.....

.....

.....

konzulens (dátum, aláírás):

.....

.....

.....

- ### 3. A szakdolgozat beadható:

.....
dátum

.....
témavezető(k)

4. A szakdolgozat szövegoldalt
 program protokollt (listát, felhasználói leírást)
 elektronikus adathordozót (részletezve)

 egyéb mellékletet (részletezve)

tartalmaz.

.....
dátum

.....
témavezető(k)

5. A szakdolgozat bírálatra bocsátható
nem bocsátható

A bíráló neve:

.....
dátum

.....
szakfelelős

- ## 6. A szakdolgozat osztályzata

a témavezető javaslata:

a bíráló javaslata:

a szakdolgozat végleges eredménye:

Miskolc,

.....
a Záróvizsga Bizottság Elnöke

Tartalomjegyzék

1. Bevezetés	7
2. Konceptió és az ökoszisztéma felépítése	8
2.1. Az EduTower koncepció	8
2.2. Háromrétegű architektúra	9
2.3. LTI integráció és konfiguráció	10
3. E-learning környezet és az LTI protokoll	11
3.1. A Moodle keretrendszer az oktatásban	11
3.2. Interaktív tananyagfejlesztés H5P környezetben	11
3.3. Az LTI protokoll és a technikai integráció	13
3.3.1. Az LTI architektúra szereplői	13
3.3.2. Az integráció és a pontszám-visszaküldés folyamata	13
4. A Unity játékmotor	15
4.1. A választott keretrendszer bemutatása	15
4.2. Összehasonlítás és technológiai indoklás	15
4.3. Felhasználói felület és technológiai kiegészítők	16
5. Fejlesztés a Unityben	17
5.1. A projekt felépítése és mappaszerkezete	17
5.2. Jelenetkezelés és hierarchia	18
5.3. Programozási logika és Manager-architektúra	19
5.4. Modularitás és bővíthetőségi lehetőségek	19
6. Code Siege – a játék bemutatása	20
6.1. Játékmechanika és célkitűzések	20
6.2. Karakterosztályok és harcrendszer	21
6.3. Az interfész és a felhasználói élmény	21
6.3.1. Az interaktív tanító és a feladatok kezelése	22
6.3.2. Kódolási ablak és visszacsatolás	22

6.4. Jutalmazási rendszer és statisztika	24
7. Fejlesztői dokumentáció	25
7.1. Szoftverarchitektúra és a Manager-rendszer	25
7.1.1. A kódkiértékelés elméleti kihívásai WebGL környezetben	26
7.1.2. A CodingManager működési logikája: Jint és Regex	26
7.1.3. Egységkezelés és vizuális visszacsatolás	27
7.1.4. A MenuManager és a dinamikus szintinicializáció	28
7.1.5. A TutorialManager és a didaktikai támogatás	29
7.1.6. Kiegészítő rendszerek és a játék lezárása	29
7.2. Interfész-alapú tervezés és az IDamageable	30
7.3. Entitások és a harcrendszer logikája	31
7.3.1. Az entitások alapja: BaseEntity	31
7.3.2. A harci egységek (Unit) működése	31
7.3.3. Lövedék mechanika és a tornyok	32
7.3.4. Adatvezérelt szintkezelés (LevelData és Resources.Load)	32
7.4. Pontozási rendszer (ScoreManager)	34
7.5. Audiovizuális elemek és erőforrások	35
8. Szerveroldali megvalósítás és integráció	36
8.1. Alkalmazott technológiák és architektúra	36
8.2. A server.js felépítése és az LTI logika	37
8.3. Adat-visszacsatolás és pontszám-szinkronizáció	37
9. Mesterséges intelligencia a játékban	39
9.1. Könnyű integráció és a Synaptic.js	39
9.2. Jelenlegi állapot és konzolos visszacsatolás	40
9.3. A jövőbeli fejlődés iránya	40
10. Felhasználói élmény és dokumentáció	41
10.1. Célcsoport és alkalmazhatóság	41
10.2. Az interaktív tanulási út	41
10.3. Integrált értékelés és visszacsatolás	42
11. Bővítési tervek és jövőkép	44
11.1. Komplex, MI-alapú kiértékelő rendszer	44
11.2. Infrastrukturális modernizáció	44
11.3. Tartalmi bővítés: szintek és karakterosztályok	45
11.4. Mobil kompatibilitás és platformbővítés	45
11.5. Felhasználói élmény (Quality of Life) fejlesztések	45

12. Összegzés és konklúzió	46
12.1. Az ökoszisztéma működése és a megoldott problémák	46
12.2. Létjogosultság a modern oktatásban	46
12.3. Szakmai fejlődés és technológiai tapasztalatok	47
13. Summary and Conclusion	48
13.1. The Ecosystem in Operation and Problems Solved	48
13.2. Relevance in Modern Education	48
13.3. Professional Growth and Technological Experiences	49
Irodalomjegyzék	50

1. fejezet

Bevezetés

Az oktatás a modern társadalmak alappillére. Az emberiség fejlődésében már a kezdetektől fogva óriási szerepet játszott a tanítás folyamata, melynek lényege évezredek óta változatlan: a tudás lehető leghatékonyabb átadása. A cél tehát ugyanaz, mint az ókori filozófusok vagy a középkori céhes inasok idején, az oktatáshoz használt eszköztárnak azonban folyamatosan alkalmazkodnia kell a kor adta lehetőségekhez. A mai világban – különösen egy olyan területen, mint a programozás – a fejlődés eddig soha nem látott sebességet diktál, mégis gyakran tapasztalható, hogy a technológiai lehetőségek és a tényleges módszertan között szakadék tátong.

Mivel ezt releváns problémának tartom, szakdolgozatom témájául az objektumorientált programozás oktatásának módszertani fejlesztését választottam. Ma már kevés fiatalot köt le a több száz oldalas dokumentáció vagy a száraz programozási tankönyv. Az objektumorientált elveken alapuló nyelvek manapság mindenhol jelen vannak – legyen szó egy játék C# kódjáról, egy alkalmazás Python scriptjéről vagy egy Java nyelven írt programról –, ezért különösen fontos, hogy ezt a témakört a lehető legerőteljebben adjuk át a jövő programozóinak.

Dolgozatom egy három fő elemből álló, interaktív oktatási ökoszisztémát mutat be. Az első elem egy Moodle-alapú, interaktív e-learning tananyag, amely az objektumorientált programozás tárgykörét követi végig fejezetről fejezetre. A fejezetek rövid elméleti összefoglalókat, valamint azokat követő interaktív feladatokat és teszteket tartalmaznak. A következő egység egy Unity környezetben készített játék, amelyben a haladáshoz a tanulóknak programozói tudásukat kell alkalmazniuk. A harmadik, összekötő elem egy közös pontrendszer, amelynek segítségével értékelhető a diákok előrehaladása. A pontrendszer visszajelzési képességeit támogatja egy a szerver kódjába beépített neurális háló, amely mesterséges intelligenciát használva, a játékban elért pontok alapján ad visszajelzést a tanulónak. Ez az azonnali visszacsatolás ösztönzőleg hat a tanulási folyamatra.

A szakdolgozat első felében az elméleti háttérrel és az alapkoncepciót fejtem ki, majd a részletes fejlesztési dokumentációt és a pontos technikai részleteket ismertetem.

2. fejezet

Konceptió és az ökoszisztéma felépítése

Ebben a fejezetben részletesen ismertetem az EduTower Defense projekt alapjául szolgáló koncepciót, a célkitűzéseket, a rendszer architektúráját, valamint az egyes komponensek közötti adatforgalom elméleti hátterét.

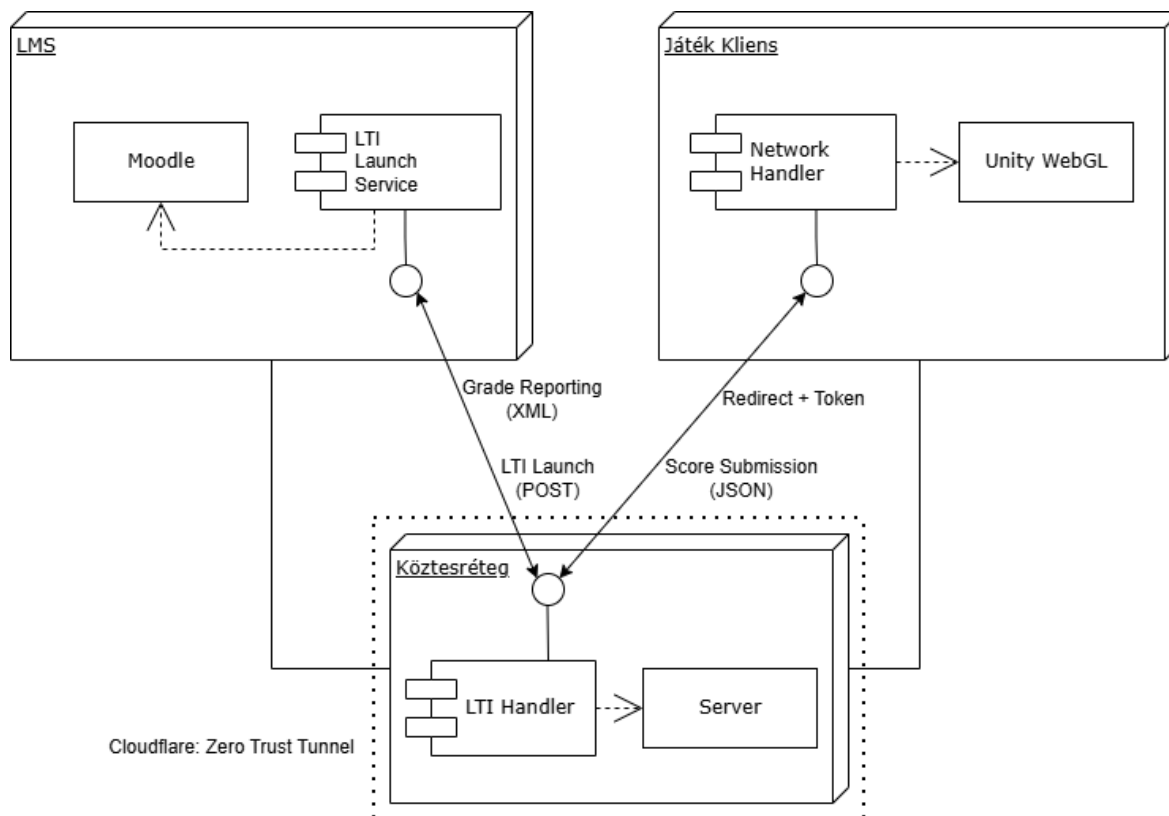
2.1. Az EduTower koncepció

A projekt alapgondolata a gamification és az automatizált visszacsatolás ötvözése az oktatásban. A fejlesztés elsődleges célja, hogy az objektumorientált programozást (OOP) tanuló hallgatók számára egy interaktív, élményközpontú tanulási környezetet biztosítson, amely ösztönzi a gyakorlati ismeretek elsajátítását. A hallgatók a böngészőben futó Unity játékban alkalmazzák elméleti tudásukat, az elért eredmény pedig az LTI protokoll segítségével közvetlenül és automatikusan megjelenik a Moodle osztályozónaplóban.

A rendszer részét képezi egy JavaScript alapú neurális háló is, amely a játékbeli teljesítmény alapján intelligens visszajelzést ad a tanulónak, ezzel támogatva az egyéni fejlődést. Bár a prototípus az OOP oktatására fókuszál, az ökoszisztéma moduláris felépítése lehetővé teszi annak későbbi adaptálását más tudományterületeken, vagy akár középiskolai szintű oktatásban is, különböző típusú interaktív tananyagok integrálásával.

2.2. Háromrétegű architektúra

A rendszer stabilitását és az interoperabilitást egy háromrétegű felépítés biztosítja, amely a 2.1. ábrán látható módon különíti el a logikai egységeket és az adatáramlási irányokat.



2.1. ábra. Az EduTower ökoszisztéma háromrétegű architektúrája és az LTI-alapú adatáramlás

A felépítés három fő komponense a következő:

- **Kliensoldali réteg (Unity WebGL):** A felhasználói interakció, a játéklogika és a kód-kiértékelés helyszíne.
- **Köztesréteg (Node.js szerver):** A rendszer „hídja”, amely az LTI hitelesítésért, az OAuth 1.0a aláírásokért és a két fő végpont közötti adatkapcsolatért felel.
- **LMS réteg (Moodle):** Az e-learning keretrendszer, amely a kurzusmenedzsmentet, a felhasználói adatokat és a központosított eredménynaplót biztosítja.

2.3. LTI integráció és konfiguráció

A Moodle és a külső eszköz közötti hitelesített kapcsolatot az LTI 1.0/1.1 protokoll biztosítja. Az integrációhoz a Moodle felületén egy „External Tool” (Külső eszköz) konfigurálása szükséges. A biztonságos adatcseréhez az alábbi paraméterek megadása kötelező:

- **Eszköz neve és webcíme:** A köztesréteg (Node.js) URL-je (`https://bmedutower.space/lti`).
- **Fogyasztókulcs (Consumer Key) és Megosztott titkos jel (Shared Secret):** Ezek a kulcspárok biztosítják a bizalmas és hitelesített kapcsolatot a két rendszer között.
- **Adatvédelmi beállítások (Privacy settings):** Annak érdekében, hogy a rendszer megfelelően működjön, engedélyezni kell, hogy az eszköz átvegye a felhasználó szükséges adatait, valamint jogosult legyen az értékelések (grades) visszaküldésére.

A konfiguráció során az IMS LTI szolgáltatások engedélyezésével válik lehetővé, hogy a játék végén keletkező pontszámok emberi beavatkozás nélkül szinkronizálódjanak a Moodle osztályozónaplójával.

3. fejezet

E-learning környezet és az LTI protokoll

Ebben a fejezetben az alkalmazott oktatási keretrendszert, a Moodle-t, a tananyag módszertani felépítését, valamint a külső eszközök integrációját biztosító LTI szabvány részletes elméleti háttérét mutatom be.

3.1. A Moodle keretrendszer az oktatásban

A Moodle (Modular Object-Oriented Dynamic Learning Environment) egy nyílt forráskódú keretrendszer, amely a modern egyetemi oktatás de facto szabványává vált [13]. Projekt-munkám során a Moodle nem csupán statikus adattárként funkcionál, hanem egy komplex oktatási ökoszisztéma központjaként, amely a teljes féléves C# objektumorientált programozás tantárgy tematikáját lefedi.

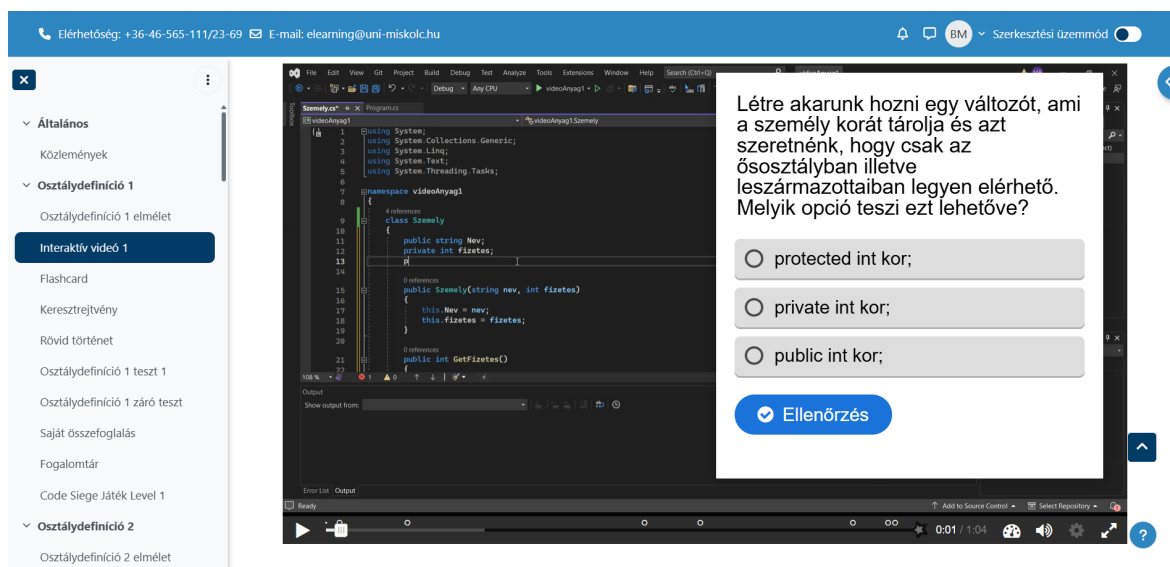
A kurzus fejezetről fejezetre haladva vezeti be a hallgatókat az OOP világába, az alapvető osztály-objektum kapcsolatoktól kezdve az öröklődésen és polimorfizmuson át a komplex tervezési mintákig. A rendszer egyik legfőbb előnye, hogy képes kezelni a hallgatói jogosultságokat, nyomon követni az előrehaladást, és egyetlen központi osztályozónaplóban egyesíteni a különböző forrásokból (H5P feladatok, tesztek, Unity játék) érkező eredményeket.

3.2. Interaktív tananyagfejlesztés H5P környezetben

A hallgatói elköteleződés növelése érdekében a statikus szövegeket interaktív elemekkel váltottam fel. A H5P (HTML5 Package)[8] keretrendszer lehetővé tette, hogy a tananyag ne csupán olvasmány, hanem aktív tevékenység legyen. Az alábbi interaktív tartalomtípusokat integráltam a rendszerbe:

- **Fogalomtárral összekapcsolt elmélet:** A rövid elméleti összefoglalókban minden kulcskifejezés (pl. *encapsulation*, *abstraction*, *interface*) aktív linkként funkcionál, amelyre kattintva a hallgató azonnal a Moodle foglomtárának releváns magyarázatához jut.
- **Gamifikált elemek:** Keresztrejtvények és Flash card-ok segítik a szintaxis és a szak kifejezések rögzítését.
- **Interaktív videók:** Megoldott kódolási feladatokról készített videókban beágyazott megállító pontok, ahol a hallgatónak kérdésekre kell válaszolnia a továbblépéshez.
- **Interaktív történet (Branching Scenario):** Valós fejlesztői döntési helyzeteket szimuláló modul, ahol a hallgató döntései befolyásolják a történet kimenetelét.
- **Rövid és hosszú tesztek:** A tudásellenőrzés skálázható. A rövid tesztek egyszerű feleletválasztós (ABC) kérdéseket vagy hiányos kódsorok kiegészítését tartalmazzák, valamint egyéb klasszikus teszt feladatokat. A hosszú tesztek ezzel szemben egy hosszabb, hiányos kódrészlet kiegészítéséből állnak.

A tananyag csúcspontja a bizonyos fejezetek végén elhelyezett **Code Siege** játék, amely az adott szakaszban megszerzett elméleti tudást ülteti át gyakorlati, szimulált környezetbe.



3.1. ábra. Interaktív videós tananyag elméleti ellenőrző ponttal a Moodle rendszerben

3.3. Az LTI protokoll és a technikai integráció

A rendszer technológiai alapköve a *Learning Tools Interoperability* (LTI) szabvány[7], amely lehetővé teszi a biztonságos adatcserét a Moodle és a külső Node.js szerver között. Az LTI 1.1 verzióját alkalmaztam, mivel ez biztosítja a legszélesebb körű kompatibilitást az osztályozónaplók automatizált kezeléséhez.

3.3.1. Az LTI architektúra szereplői

Bár az LTI 1.1[16] szabvány formálisan két logikai szerepkört definiál, a gyakorlati megvalósítás során – a rendszer háromrétegű architektúrájából adódóan – ez a feladatkör megoszlik a köztesréteg és a kliensoldal között. A Moodle szempontjából a teljes külső ökoszisztéma egyetlen egységként jelenik meg.

- **LTI Tool Consumer (LMS):** Az oktatási keretrendszer (Moodle), amely elindítja a kérést, kezeli a hallgatói munkamenetet, és végül befogadja az osztályzatokat.
- **LTI Tool Provider (Middleware és Kliens):** A szabvány szerinti „szolgáltató” szerepét projektünkben a Node.js szerver és a Unity WebGL játék együttesen látja el.
 - A *Node.js* szerver funkcionál a hitelesítési kapuként: ez fogadja a Moodle kéréseit, kezeli az OAuth aláírásokat és generálja a visszaküldendő XML állományokat.
 - A *Unity játék* pedig a tényleges tartalomszolgáltató, amely a Node.js szerveren keresztül kapja meg a szükséges azonosítókat, majd a folyamat végén oda küldi vissza a nyers pontszámokat.

3.3.2. Az integráció és a pontszám-visszaküldés folyamata

A biztonságos kommunikáció alapja az OAuth 1.0a protokoll. A Moodle egy digitálisan aláírt POST kérést (Launch Request) küld a `https://bmedutower.space/lti` végpontra. Ez a kérés tartalmazza:

- A hallgató anonimизált azonosítóját (`user_id`).
- A hitelesítéshez szükséges `oauth_consumer_key`-t.
- Az `lis_outcome_service_url` paramétert, amely megadja, hová kell a szervernek visszaküldenie a pontszámot.
- A `lis_result_sourcedid` azonosítót, amely az adott hallgató adott feladathoz tartozó egyedi bejegyzését azonosítja a Moodle adatbázisában.

Amikor a diák befejezi a játékot, a Node.js szerver egy `imsx_POXEnvelopeRequest` típusú XML üzenetet generál. Ebben a 0 és 100 közötti játékbeli pontszámot egy 0.0 és 1.0 közötti lebegőpontos értéké transzformálja, majd ezt az értéket a `resultScore` elemen belüli `textString` tagben helyezi el. Az üzenetet a szerver HMAC-SHA1 algoritmus-sal és OAuth aláírással hitelesítve küldi vissza a Moodle `outcomeUrl` végpontjára. Ezzel valósul meg a teljesen emberi beavatkozástól mentes, automatizált és hitelesített osztályozás.

4. fejezet

A Unity játékmotor



Ebben a fejezetben röviden ismertetem a választott fejlesztői környezetet, a Unity [28] motor sajátosságait, valamint a döntésemet alátámasztó technológiai érveket.

4.1. A választott keretrendszer bemutatása

A fejlesztéshez a **Unity 6.3 LTS (6000.3.9f1)** verzióját választottam. A Unity egy széles körben elterjedt, **platformfüggetlen** játékmotor, amely kiemelkedő támogatást nyújt a **WebGL** [27] alapú build-ek készítéséhez. Ez a funkció elengedhetetlen volt a projekt szempontjából, hiszen a cél egy böngészőben futó, telepítést nem igénylő oktatóprogram létrehozása volt, amely zökkenőmentesen integrálható a Moodle keretrendszerbe.

A motor egyik legnagyobb előnye a C# nyelv natív támogatása, amely lehetővé teszi a magasszintű, objektumorientált programozást a kliensoldali logika fejlesztésekor. Emellett a Unity gazdag Asset Store kínálata és kiterjedt dokumentációja jelentősen felgyorsította a fejlesztési folyamatot.

4.2. Összehasonlítás és technológiai indoklás

A fejlesztési környezet kiválasztásakor több alternatívát is mérlegeltem (pl. Unreal Engine, Godot), azonban a Unity bizonyult a legoptimálisabb megoldásnak az alábbi szempontok alapján:

- **Gépigény és hatékonyság:** Az Unreal Engine-nel szemben, amely elsősorban a magas hardverigényű, komplex 3D grafikára fókuszál, a Unity rugalmasabb és kevésbé gépigényes környezetet biztosít a 2D-s projektekhez, anélkül, hogy kompromisszumot kellene kötni a vizualitás terén.
- **Komplexitás és támogatottság:** A Godot motorral összehasonlítva a Unity kiforrottabb asset-könyvtárat és szélesebb körű kiegészítőket kínál, ami egy szakdolgozati szintű, összetettebb rendszernél elengedhetetlen volt a professzionális végeredmény érdekében.
- **Keresztplatformos lehetőségek:** A Unity lehetővé teszi, hogy ugyanazt a forráskódot minimális módosítással asztali vagy mobil platformokra is exportálhassuk. Ez a rugalmasság fontos szempont volt a tesztelési fázisban és a jövőbeli bővíthetőség tekintetében.

4.3. Felhasználói felület és technológiai kiegészítők

A játék vizuális megjelenéséért és a felhasználói interakciókért a Unity modern **UI Toolkit** [26] rendszere felel. A felületek kialakításánál törekedtem a legelterjedtebb képernyőarányok szerinti optimális megjelenítésre.

A szöveges tartalom és a kód megjelenítés minőségét a **TextMesh Pro** csomag biztosítja. A kliensoldali kódkiértékelés folyamatát a **Jint** [17] interpreter és reguláris kifejezések kombinációja látja el, amely lehetővé teszi a felhasználó által írt kódok dinamikus elemzését.

5. fejezet

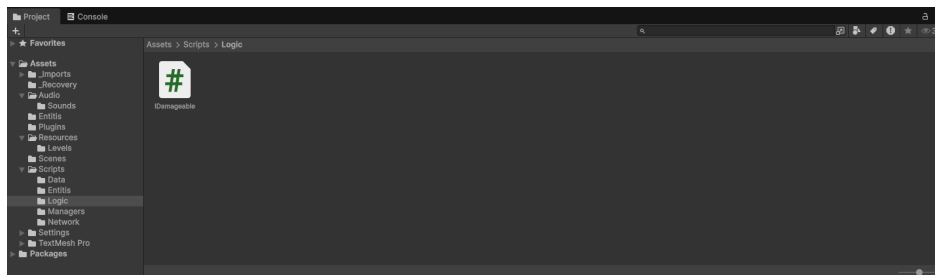
Fejlesztés a Unityben

Ebben a fejezetben a projekt belső architektúráját, a Unity-n belüli erőforrás-kezelést, valamint a rendszer modularitását és bővíthetőségét mutatom be.

5.1. A projekt felépítése és mappaszerkezete

A hatékony fejlesztés és a hosszú távú karbantarthatóság alapja a projekt szigorú hierarchikus szervezettsége [23]. Az **Assets** gyökérkönyvtár alatti felosztás biztosítja, hogy a különböző típusú erőforrások logikailag elkülönüljenek:

- **_Imports:** Ebben a mappában kaptak helyet a külső forrásból származó assetek, modellek és textúrák, elkülönítve azokat a saját fejlesztésű elemektől.
- **Audio:** Itt találhatók a zenei sávok és hangeffektek, valamint a hangforrások kezeléséhez szükséges objektumok.
- **Entities:** A játékmenet „testét” adó *Prefabbok* gyűjtőhelye. Itt kaptak helyet az egységek (**Archer**, **Knight**, **Enemy**), a lövedékek és a vizuális visszajelzők, mint a **HealthBar**.
- **Plugins:** A projekt motorjaként funkcionáló külső könyvtárak helye. Itt található a **Jint** interpreter, amely a diákok JavaScript kódjának futtatásáért felel.
- **Resources/Levels:** A projekt adatvezérelt jellegének kulcshelyszíne. Itt tárolódnak a pályák paramétereit leíró állományok, amelyeket a rendszer futásidőben olvas be.
- **Scripts:** A projekt „agya”. A C# forráskódokat funkcionális alkönyvtárakba (**Data**, **Logic**, **Managers**, **Network**) rendeztem a könnyebb átláthatóság érdekében.



5.1. ábra. Az absztrakciót segítő interfészek elhelyezése a projektstruktúrában

5.2. Jelenetkezelés és hierarchia

A projekt két fő jelenetre (**Scene**) [24] oszlik: a **Menu** felel az indításért és a globális beállításokért, míg a **SampleScene** a dinamikus játéktér helyszíne. A hierarchia felépítése során törekedtem a tiszta elkülönítésre: az **ENVIRONMENT** a vizuális világot, a **Canvas** a UI elemeket, míg a **SYSTEM** a háttérfolyamatokat koordinálja.



5.2. ábra. A hierarchia ablak

5.3. Programozási logika és Manager-architektúra

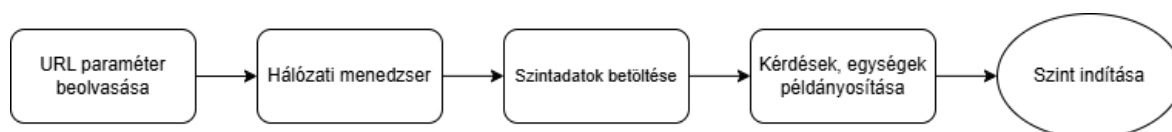
A rendszer vezérlését dedikált központi szkriptek, úgynevezett **Managerek** végzik. Ez a megközelítés lehetővé teszi, hogy a funkciók elszeparáltak és jól tesztelhetően működjenek.

5.4. Modularitás és bővíthetőségi lehetőségek

A szoftver tervezésekor kiemelt szempont volt a könnyű módosíthatóság és a skálázhatóság. A rendszer architektúrája két szinten is támogatja a bővítést:

1. Adatvezérelt pályabővítés: A játékmenet nem „bedrótozott” (hardcoded) szintekből áll. A **LevelData.cs** struktúra és a dinamikus betöltési logika lehetővé teszi, hogy új pályákat hozzunk létre pusztán az adatfájlok bővítésével. Így az oktatónak nem kell módosítania a forráskódot, ha új feladatot szeretne a tananyagba illeszteni.

2. Új egységek implementálása: Az **IDamageable** interfész [11] és a **Prefab** [22] rendszer használata biztosítja, hogy új típusú katonák vagy ellenségek (például egy varázsló egység) minimális fejlesztéssel hozzáadhatók legyenek. Az életerő-kezelés és a kódkiértékelés rendszerei készen állnak az új entitások kiszolgálására is.



5.3. ábra. A szintbetöltés logikája

6. fejezet

Code Siege – a játék bemutatása

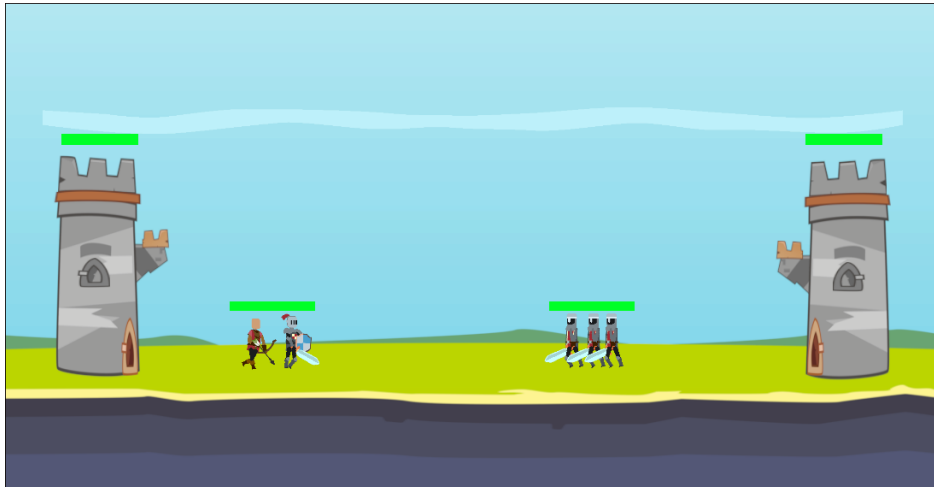
Ebben a fejezetben az EduTower projekt központi elemét, a Code Siege játékmenetét, szabályrendszerét és technikai megvalósítását részletezem. A játék célja, hogy a hallgatók egy szimulált csatamezőn, gyakorlati feladatokon keresztül mélyítsék el programozói ismereteiket.

6.1. Játékmechanika és célkitűzések

A Code Siege egy 2D-s oldalnézetes (side-scroller) Tower Defense és Offense (toronyvédelem és ostrom) elemeit vegyítő játék. A játékos elsődleges célja az ellenfél tornyának lerombolása saját egységek segítségével.

A játékmenet ciklikus felépítésű:

1. **Feladatmegoldás:** Minden szint öt programozási feladatból áll. A hallgató minden sikeresen megoldott feladat után egy katonát kap jutalmul.
2. **Seregszemle:** Egy szinten maximum öt egység szerezhető meg. Hiába igényel a feladat (például egy ciklus segítségével) több egység leidézését, a rendszer minden lezárt feladatért egy konkrét entitást delegál a későbbi harcba.
3. **Ütközet:** Miután a hallgató tesztelte és jóváhagyta kódjait, megkezdődik a csata. Az egységek automatikusan mozognak és harcolnak az ellenfél szintenként növekvő számú katonáival szemben.
4. **Kiértékelés:** A győzelem feltétele az ellenséges torony életerejének nullára csökkentése. Vereség esetén a saját torony pusztulása jelzi a kudarcot.



6.1. ábra. Code Siege harc fázis

6.2. Karakterosztályok és harcrendszer

A játék jelenleg két különböző egységtípust különböztet meg, amelyek taktikai szerepe eltérő:

- **Lovag (Knight):** Az alapvető közelharcos egység, amely az első szinttől elérhető. Magasabb életerővel rendelkezik, feladata az arcvonal megtartása és a sebzés elnyelése. Leidézése a `lovag()`; függvényhívással történik.
- **Íjász (Archer):** A második szinttől bevezetett távolsági egység. Alacsonyabb életerővel bír, de képes messziről sebezni az ellenséget. A diák az `ijasz()`; hívással (bizonyos esetekben paraméterezve, pl. `ijasz(50);`) hívhatja segítségül.

A karakterek attribútumai (életerő, sebzés, támadási sebesség) karakterenként eltérőek, továbbá a rendszer megkülönbözteti az ellenséges és a barátságos egységek értékeit is az egyensúly fenntartása érdekében.

6.3. Az interfész és a felhasználói élmény

A felhasználói felületet úgy alakítottam ki, hogy az támogassa a tanulási folyamatot. A játék a menürendszerrel indul, ahol a hangbeállítások és az információs panel mellett látható az aktuális szint kiválasztása.



6.2. ábra. A Code Siege menü panele

6.3.1. Az interaktív tanító és a feladatok kezelése

A szint indításakor egy animált középkori tanító karakter vezeti be a hallgatót a kihívásba. A tanító szövegbuborékban közli a feladatokat, majd a háttérbe húzódva folyamatos segítséget nyújt az információ gombon keresztül. Az interfész felső részén található funkciógombok (F1, F2 stb.) segítségével a hallgató bármikor válthat a feladatok között, ilyenkor a tanító mindig az adott feladathoz tartozó segítő szöveget és kérdést ismerteti.



6.3. ábra. A Code Siege tutor panele

6.3.2. Kódolási ablak és visszacsatolás

A kódolási panel központi eleme a szerkesztőablak, amely felett a feladat rövidített leírása, alatta pedig a futtatáshoz szükséges vezérlők találhatók:

- **Infó:** Előhívja a tanító részletes segítő szövegét.
- **Teszt:** Ellenőrzi a kódot. A szintaktikai vagy logikai eredményről egy külön szöveges sáv ad visszajelzést. Sikeres teszt esetén a kód zárolásra kerül.
- **Start:** Elindítja a szimulációt és az ütközetet.



6.4. ábra. A Code Siege kódolási panele

A rendszer intelligens hibaüzenetekkel segíti a hallgatót: a tanító a konkrét hibakód alapján specifikus visszajelzést ad, ha a beküldött megoldás nem megfelelő.



6.5. ábra. A Code Siege hibaüzenete

6.4. Jutalmazási rendszer és statisztika

A motiváció fenntartásáért egy aranyalapú gazdasági rendszer felel. Minden sikeresen teljesített szint egy aranyat ér, amely közvetlenül befolyásolja a hallgató végső pontszámát. Az ütközet végén egy összefoglaló panel jelenik meg, amely a győzelem vagy vereség ténye mellett mutatja a megszerzett pontokat és az aktuális aranykészletet, amelyet az LTI protokollon keresztül szinkronizálunk a Moodle rendszerrel.



6.6. ábra. A Code Siege "játék vége" panele

7. fejezet

Fejlesztői dokumentáció

Ebben a fejezetben az EduTower projekt technikai megvalósítását, a szoftverfejlesztési döntések háttérét és a rendszer belső működését mutatom be részletesen. A cél a „motorháztető alatti” megoldások transzparens ismertetése, kitérve az alkalmazott tervezési mintákra, az objektumorientált megközelítésekre, valamint a külső könyvtárak és technológiák integrációjának módjára. A dokumentáció ezen része igazolja a rendszer modularitását és a hosszú távú fenntarthatóságot szolgáló precíz tervezést.

7.1. Szoftverarchitektúra és a Manager-rendszer

A projekt fejlesztése során a Unity környezetben elterjedt Manager-mintát alkalmaztam, amely lehetővé teszi a logika központosítását és a felelősségi körök tiszta elkülönítését [23]. A rendszer gerincét dedikált menedzser osztályok alkotják, amelyek koordinálják a játékmenetet, a felhasználói felületet és az adatforgalmat. A táblázatban a fejezet ezen részéhez szorosabban fűződő osztályok nevei és általános feladatai láthatók.

Osztály neve	Felelősségi kör
CodingManager	A játék központi vezérlője. Felel a feladatok kezeléséért, a Jint motor inicializálásáért, a kódellenőrzésért és az egységek példányosításáért.
MenuManager	Kezeli a kezdőképernyőt, az URL-alapú szintválasztást és a globális beállítások elérését.
TutorialManager	Az animált tanító karakter mozgását (Coroutine-alapú elmozdulás) és a dialógusbuborékok tartalmát vezérli.
ScoreManager	A hallgatói teljesítmény (eltelt idő és megszerzett arany) alapján számítja ki a 0 és 100 közötti végső pontszámot.
GameOverManager	A játék végén megjelenő statisztikai panel inicializálását és a játék újraindítását végzi.

7.1. táblázat. A szoftverarchitektúra központi menedzser osztályai

7.1.1. A kódkiértékelés elméleti kihívásai WebGL környezetben

A *Code Siege* legfontosabb mechanikája a hallgató által beírt forráskód valós idejű kiértékelése. Ez a folyamat informatikai szempontból egy „kód a kódban” szituációt eredményez, amelynek megvalósítása a választott WebGL platform miatt jelentős akadályokba ütközik.

A C# fordítási folyamata és a JIT hiánya A C# alapvetően egy erősen típusos, fordított nyelv. Normál asztali környezetben a forráskód először egy köztes nyelvre (CIL) fordul, majd futtatáskor a *Just-In-Time* (JIT) fordító alakítja át gépi kóddá. A WebGL platformon azonban a böngészők biztonsági architektúrája nem teszi lehetővé a futásidejű kódgenerálást. A Unity ezt az **IL2CPP** [21] technológiával hidalja át, amely a teljes projektet még a futtatás előtt natív C++ kódra, majd WebAssembly-re fordítja (*Ahead-of-Time* fordítás). Ezért a standard C# reflexiós és dinamikus kódgenerálási mechanizmusok ezen a platformon nem használhatóak.

Alternatívák és a döntés háttere A fejlesztés során vizsgáltam a Roslyn fordító integrálását, azonban az a bájtkód-generálás hiánya és a jelentős fájl méret miatt WebGL alatt nem alkalmazható. A Lua-alapú interpreterek (pl. MoonSharp) bár stabilak, szintaktikailag túlságosan eltérnek a C# nyelvtől, ami megzavarná az oktatási folyamatot. A választott megoldás végül a **Jint** [17] (JavaScript interpreter) és a **Reguláris Kifejezések** (Regex) [12] kombinációja lett, amely lehetővé teszi a C# szintaxis szimulációját egy böngésző-kompatibilis környezetben.

7.1.2. A CodingManager működési logikája: Jint és Regex

A kódkiértékelő rendszer kétlépcsős folyamatban dolgozza fel a hallgatói bemenetet, amelyet a CodingManager osztály vezérel.

A Jint interpreter és a típusmentesítés A Jint egy .NET-re írt JavaScript interpreter, amely az absztrakt szintaxis fán (AST) keresztül értelmezi a kódot, így nem igényel bájtkód-generálást. Mivel a Jint alapvetően JavaScriptet vár, a hallgató által beírt C# kódot futtatás előtt át kell alakítani. A `PreprocessCodeForJint` metódus reguláris kifejezéseket használ a típusmentesítésre: az összes C# típusmegnevezést (például `int`, `string`, `float`) a JavaScript `var` kulcsszavára cseréli.

Pedagógiai hibaüzenet-fordítás A kód futtatása egy `try-catch` blokkban történik. Ha esetén a `TranslateErrorMessage` metódus elemzi a Jint által küldött technikai üzenetet, és a hallgató számára érthető segítő szöveggé alakítja azt.

```

private string PreprocessCodeForJint(string rawCode) {
    return Regex.Replace(rawCode, @"\"b(int|float|double|string|bool|char)
        \b", "var");
}

private string TranslateErrorMessage(string rawMessage) {
    if (rawMessage.Contains("is not defined"))
        return "Olyan nevet hasznaltal, amit nem tanultunk meg!";
    if (rawMessage.Contains("Unexpected token"))
        return "Ugy latom, elfelejtettel egy irasjelet (zarojelek,
            pontosvesszok?)";
    return "Sajnos hibas a kod: " + rawMessage;
}

```

Kódrészlet 7.1. A típusok konverziója és a hibaiüzenetek kezelése a CodingManager-ben

7.1.3. Egységkezelés és vizuális visszacsatolás

A CodingManager feladata a sikeres feladatmegoldások fizikai jutalomá alakítása a játéktérben.

A csata inicializálása és egységgyártás A StartBattle metódus meghívásakor a rendszer kilép a kódoló felületről és aktiválja a harci fázist. Ekkor a playerArmy listában tárolt Prefab hivatkozások alapján a rendszer példányosítja a hallgató által megszerzett egységeket. Az alábbi kódrészlet szemlélteti, hogyan kerülnek az egységek a virtuális térbe a megfelelő spawn-pontokon, életerőcsíkokkal kiegészítve:

```

private void SpawnPlayerArmyWithHealthBar() {
    int unitsToSpawn = Mathf.Min(playerArmy.Count, knightSpawnPoints.Count);

    for (int i = 0; i < unitsToSpawn; i++) {
        GameObject prefabToSpawn = playerArmy[i];

        GameObject newUnit = Instantiate(prefabToSpawn, knightSpawnPoints[i].position, Quaternion.Euler(0, 180, 0));

        Vector3 healthBarPosition = newUnit.transform.position + new Vector3(0, 1.4f, 0);
        GameObject newBar = Instantiate(healthBar, healthBarPosition, Quaternion.identity);
        newBar.transform.SetParent(newUnit.transform);

        Unit unitScript = newUnit.GetComponent<Unit>();
        unitScript.healthFillImage = newBar.transform.Find("Fill").GetComponent<Image>();
    }
}

```

Kódrészlet 7.2. A hallgatói sereg példányosítása és az életerőcsíkok inicializálása

7.1.4. A MenuManager és a dinamikus szintinicializáció

A szoftver futtatásakor a MenuManager az első aktív komponens, amely felelős a futtatási környezet felméréséért és a megfelelő játékszint előkészítéséért. Feladata túlmutat a hagyományos menürendszerek vezérlésén: ez az osztály teremti meg a technikai kapcsolatot a külső keretrendszer és a belső játékmotor között.

URL-alapú paraméterezés és adatátvitel A projekt egyik legfontosabb technikai követelménye, hogy a hallgató közvetlenül a keretrendszerből (például Moodle-ból) indíthassa el az aktuális feladatsort. Ezt a MenuManager az `Application.absoluteURL` tulajdonság lekérdezésével és elemzésével valósítja meg. A `Start` metódus lefutásakor a rendszer megvizsgálja, hogy az URL tartalmazza-e a `?level=` paramétert. Amennyiben igen, a szkript string-manipulációs eljárásokkal izolálja a szint azonosítóját, majd ezt egy statikus változóban (`LevelToLoad`) tárolja el. Ez a megoldás teszi lehetővé, hogy a Unity jelenetváltáskor is megőrizze az információt.

Hibatűrés és alapértelmezett állapotok Annak érdekében, hogy a szoftver paraméterek hiányában is működőképes maradjon, implementálásra került a `LoadDefault` metódus. Amennyiben az URL elemzése sikertelen, vagy nem található benne szintmeghatározás, a rendszer automatikusan a „Level_2” azonosítót rendeli hozzá a betöltési folyamathoz. Ez a redundancia biztosítja a stabil futást tesztkörnyezetben és éles bevetés során egyaránt.

```
string currentUrl = Application.absoluteURL;

if (!string.IsNullOrEmpty(currentUrl) && currentUrl.Contains("?level="))
{
    try
    {
        string[] urlParts = currentUrl.Split(new string[] { "?level=" },
            StringSplitOptions.None);
        string levelString = urlParts[1].Split('&')[0];

        if (int.TryParse(levelString, out int parsedLevel))
        {
            LevelToLoad = "Level_" + parsedLevel;
        }
    }
    catch
    {
        LoadDefault();
    }
}
```

Kódrészlet 7.3. Az URL-alapú szintszám kinyerésének implementációja

7.1.5. A TutorialManager és a didaktikai támogatás

A hallgatói élmény és az elméleti segítségnyújtás központja a TutorialManager. Az osztály feladata az animált tanító karakter viselkedésének vezérlése és a többoldalas elméleti anyagok megjelenítése.

Vizuális visszacsatolás és Coroutine-alapú animáció A tanító panel megjelenítése nem statikus módon történik; a SwimIn és SwimOut metódusok egy folyamatos beúszó animációt valósítanak meg. A technikai megvalósítás *Coroutine*-okat [20] alkalmaz, amelyek a `Vector2.Lerp` függvény segítségével, lineáris interpolációval mozgatják a panelt a megadott `anchoredPosition` értékek között. Ez a megoldás lehetővé teszi az aszinkron futást, így az animáció nem blokkolja a játék többi folyamatát.

Interaktív segítségnyújtás és lapozás A TutorialManager szorosan együttműködik a CodingManager-rel: minden új feladatnál megjeleníti a hozzá tartozó elméleti hátteret. Mivel a magyarázatok hossza változó, az osztály kihasználja a `TextMeshPro.pageToDisplay` tulajdonságát. A hallgató a felületen található gombokkal lapozhat az információk között, amit a `OnOkButtonClicked` és `OnBackButtonClicked` eseménykezelők vezérelnek.

```
private IEnumerator MovePanel(Vector2 targetPosition)
{
    float duration = 0.5f;
    float elapsedTime = 0f;
    Vector2 startPosition = rectTransform.anchoredPosition;

    while (elapsedTime < duration)
    {
        rectTransform.anchoredPosition = Vector2.Lerp(startPosition,
            targetPosition, elapsedTime / duration);
        elapsedTime += Time.unscaledDeltaTime;
        yield return null;
    }

    rectTransform.anchoredPosition = targetPosition;
}
```

Kódrészlet 7.4. A tanító panel mozgatásáért felelős enumerátor

7.1.6. Kiegészítő rendszerek és a játék lezárása

A központi vezérlés mellett több specializált menedzser felel a játékelmény teljessé tételéért, az audiovizuális visszacsatolásért és a folyamatok szabályos lezárásáért.

ScoreManager és SettingsManager A `ScoreManager` feladata a hallgatói teljesítmény számszerűsítése az összegyűjtött arany és a feladatmegoldásra fordított idő alapján. A pontszámítás egy normalizált algoritmust követ, amely a végeredményt 0 és 100 pont közé szorítja. A játékelmény testreszabását a `SettingsManager` teszi lehetővé, amely közvetlen kapcsolatot teremt a felhasználói felület csúszkái és a hangrendszer között, biztosítva a beállítások mentését a munkamenetek között.

AudioManager és a dinamikus hangkezelés A projekt hangkörnyezetéért az `AudioManager` felel, amely *Singleton* mintát követve a teljes játék alatt elérhető marad. Az osztály külön csatornákon kezeli a háttérzenét (*BGM*) és az effektusokat (*SFX*). Technikai érdekessége az automatikus eseménykezelő hozzárendelés, amely a jelenet betöltésekor minden gombhoz hozzárendeli a kattintás hangját, így mentesítve a fejlesztőt a manuális beállításoktól.

GameOverManager és a visszacsatolás A játékmenet lezárása egy többlépcsős folyamat, amely több menedzser szoros együttműködését igényli. A folyamat akkor indul el, amikor egy bázisobjektum (torony) életerejére eléri a nullát, és meghívja a `CodingManager.EndGame` metódust. Ezt követően, amikor a `CodingManager` jelzi a csata befejezését, ez az osztály aktiválja a zárófelületet. A `Setup` metódus paraméterezhető módon fogadja a győzelem vagy vereség tényét, a szerzett aranyakat és a végső pontszámot, majd ezeket vizuálisan megjeleníti a hallgatónak. Ezen kívül lehetőséget ad a játékosnak a menübe való visszatérésre a `SceneManager` segítségével.

```
public void Setup(string result, int gold, int score, Color statusColor)
{
    panel.SetActive(true);
    resultText.text = result;
    resultText.color = statusColor;
    goldText.text = gold + "G";
    scoreText.text = score + "XP";
}

public void RestartGame()
{
    Time.timeScale = 1.0f;
    SceneManager.LoadScene("Menu");
}
```

Kódrészlet 7.5. A `GameOverManager` eredményjelző inicializálása

7.2. Interfész-alapú tervezés és az `IDamageable`

A projekt kódjának rugalmasságát és karbantarthatóságát az interfész-alapú [11] tervezés biztosítja. Bár a rendszerben korlátozott számú interfész található, az `IDamageable` kulcs-

fontosságú szerepet tölt be a harcrendszer egységesítésében.

Az interfész egyfajta „szerződésként” fogható fel: meghatározza, hogy az azt megvalósító osztályoknak milyen metódusokkal kell rendelkezniük, de a konkrét implementációt az osztályokra bízta. A *Code Siege*-ben ez azért előnyös, mert a sebzést okozó lövedékeknek vagy egységeknek nem kell ismerniük a célpont pontos típusát (legyen az egy mozgó egység vagy egy statikus torony), csupán azt, hogy az objektum sebzhető-e. Ez a megközelítés csökkenti az osztályok közötti függőséget, és lehetővé teszi új sebzhető entitások hozzáadását a meglévő harci logika módosítása nélkül.

```
public interface IDamageable
{
    void TakeDamage(float damage);
    void Die();
}
```

Kódrészlet 7.6. Az IDamageable interfész definíciója

7.3. Entitások és a harcrendszer logikája

A játékmenet magját az entitások hierarchiája és a köztük zajló interakciók alkotják. A rendszer alapja egy absztrakt ősosztály, amelyből minden sebzhető objektum származik.

7.3.1. Az entitások alapja: BaseEntity

A *BaseEntity* egy absztrakt osztály, amely implementálja az *IDamageable* interfészt, és tartalmazza az összes entitás közös tulajdonságait: az életerőt (*maxHealth*, *currentHealth*), a csapataazonosítót (*teamID*) és az életerő megjelenítéséért felelős UI elemet (*healthFillImage*).

Az osztály egyik legfontosabb metódusa a *TakeDamage*, amely nem csupán az életerőt csökkenti, hanem automatikusan frissíti a felhasználói felületet is. Amennyiben az egység életeréje nullára csökken, a rendszer letiltja az ütközésvizsgálatot (*Collider2D*), megállítja a fizikai szimulációt (*Rigidbody2D*), és elindítja a halál-animációt, mielőtt véglegesen törölné az objektumot.

7.3.2. A harci egységek (Unit) működése

A *Unit* osztály a *BaseEntity* leszármazottja, amely mozgási és támadási képességekkel egészül ki. Az egységek a *teamID* alapján döntenek el a haladási irányt, és folyamatosan figyelik az útvonalukba kerülő objektumokat.

A harci érintkezés triggereken keresztül valósul meg. Ha az egység látókörébe ellenséges csapatba tartozó entitás kerül, a mozgása megáll, és elindul egy *Coroutine*-alapú támadási ciklus (*AttackRoutine*). A rendszer támogatja mind a közelharci (*Melee*), mind a

távolsági (Ranged) támadást. Közelharc esetén az egység közvetlenül meghívja a célpont `TakeDamage` metódusát, míg távolsági harcnál a rendszer egy külön lövedék objektumot (`Projectile`) példányosít.

7.3.3. Lövedék mechanika és a tornyok

A távolsági támadások vizuális és funkcionális eleme a `Projectile` osztály. A lövedékek egy íves pályát írnak le, amelyet a `Mathf.Sin` függvénnyel számított eltolás tesz látványossá. A lövedék folyamatosan követi a célpontot, és a becsapódáskor adja át a sebzést az `IDamageable` interfészen keresztül.

A `Tower` osztály speciális entitásként funkcionál: bár alapjaiban a `BaseEntity` tulajdonságait használja, halála esetén – azaz a bázis megsemmisülésekor – értesíti a `CodingManager`-t a játék végétől.

```
public virtual void TakeDamage(float damage)
{
    currentHealth -= damage;

    if (healthFillImage != null)
    {
        healthFillImage.fillAmount = currentHealth / maxHealth;
    }

    if (currentHealth <= 0)
    {
        // Komponensek leallitása és halál kezelése
        Collider2D col = GetComponent<Collider2D>();
        if (col != null) col.enabled = false;

        if (animator != null) animator.SetBool("isAlive", false);
        Invoke(nameof(Die), 1f);
    }
}
```

Kódrészlet 7.7. Az egységek életerő-kezelésének megvalósítása

7.3.4. Adatvezérelt szintkezelés (`LevelData` és `Resources.Load`)

A Code Siege rendszerének rugalmasságát és skálázhatóságát az adatvezérelt tervezés biztosítja. A játékszintek paraméterei nem a forráskódba égetve, hanem különálló adatállományokban (Asset) tárolódnak, ami lehetővé teszi a feladatok és a játékmenet módosítását a szoftver újratöltése vagy módosítása nélkül.

A megvalósítás alapja a `LevelData.cs` osztály, amely a `Unity ScriptableObject` [25] alapsztályából származik. Ez a megközelítés lehetővé teszi, hogy az adatszerkezeteket a Unity Editorban vizuálisan szerkesszük. A rendszer központja a `QuestionData` struktúra, amely egy-egy feladat minden technikai és didaktikai követelményét tartalmazza.

```

[Serializable]
public struct QuestionData
{
    [Header("Feladat adatai")]
    public string Description;
    [TextArea(3, 10)]
    public string Theory;

    [Header("Logika")]
    public int RequiredKnightCount;
    public int RequiredArcherCount;
    public int RequiredPower;
    public string RequiredVarName;
    public int ExpectedVarValue;

    [Header("Regex - Szintaktika")]
    public string RequiredRegexPattern;

    [Header("Jutalom a feladatért")]
    public UnitReward RewardUnit;
}

[CreateAssetMenu(fileName = "NewLevelData", menuName = "ScriptableObjects
/LevelData")]
public class LevelData : ScriptableObject
{
    public string LevelName;
    [Header("Palya beallitasok")]
    public int EnemyCount = 2;
    public List<QuestionData> Questions;
}

```

Kódrészlet 7.8. A LevelData és QuestionData struktúra implementációja

A QuestionData struktúra RequiresJint tulajdonsága automatikusan dönti el, hogy a hallgató kódjának ellenőrzéséhez elegendő-e a RequiredRegexPattern alapján végzett szintaktikai szűrés, vagy szükség van a Jint interpreter általi logikai kiértékelésre is.

A konkrét pályák adatait a Resources/Levels mappában elhelyezett YAML alapú asset fájlok tárolják. A Level_1.asset például a legalapvetőbb programozási lépéseket (függvényhívás, for ciklus, osztálydefiníció) tartalmazza, míg a Level_2.asset és Level_3.asset már összetettebb koncepciókat (öröklődés, láthatósági módosítók, absztrakt osztályok és kivételkezelés) definiál.

```
%YAML 1.1
%TAG !u! tag:unity3d.com,2011:
--- !u!114 &11400000
MonoBehaviour:
  m_Name: Level_1
  LevelName: Level_1
  EnemyCount: 2
  Questions:
  - Description: "Hivd meg a lovag(); fuggvenyt..."
    RequiredKnightCount: 1
    RewardUnit: 0
  - Description: "Idezz meg pontosan 3 lovagot egy for ciklussal!"
    RequiredRegexPattern: for\s*\(
    RewardUnit: 0
```

Kódrészlet 7.9. A Level_1.asset fájl szerializált struktúrája (részlet)

Mivel a szintek dinamikus betöltése a MenuManager által kapott URL paraméterek alapján történik. A rendszer a Resources.Load<LevelData> metódust használva emeli be a megfelelő adatokat a memóriába futásidőben. Ez a megoldás biztosítja, hogy a WebGL kliens a böngészőben minimális késleltetéssel, külső adatbázis-lekérések nélkül tudja inicializálni az aktuális oktatási modult.

7.4. Pontozási rendszer (ScoreManager)

A hallgatói teljesítmény számszerűsítése a ScoreManager feladata, amely a Moodle rendszer számára küldött végső százalékos értékelést kalkulálja. Az algoritmus célja, hogy ne csak a helyes megoldásokat, hanem a hatékonyságot (gyorsaságot) is díjazza.

A pontszámítás alapját a megszerzett aranyak (goldCount) adják, amelyet egy időalapú bónusz egészít ki. Az időbónusz a szint indulásától eltelt idő függvényében lineárisan csökken, ösztönözve a hallgatókat a magabiztos feladatmegoldásra. A végeredményt a rendszer 100 pontban maximalizálja.

```
public int CalculateFinalScore(int goldCount) {
    int basePoints = goldCount * 15;
    float elapsedTime = Time.time - levelStartTime;

    // Max 40 pont bonusz, amely az ido elorehaladtaval csokken
    int timeBonus = Mathf.Max(0, 40 - Mathf.RoundToInt(elapsedTime * 0.5f));

    int finalScore = basePoints + timeBonus;
    return Mathf.Min(finalScore, 100);
}
```

Kódrészlet 7.10. A pontszámítás logikai magja

7.5. Audiovizuális elemek és erőforrások

A játékban alkalmazott hangok és vizuális megoldások a felhasználói interakciók támogatását, valamint a rendszerszintű visszacsatolások megerősítését szolgálják. A hanghatások kezeléséért az `AudioManager` felel, amely elkülönített csatornákon kezeli a háttérzenéket és a hangeffekteket.

A hanganyagok beszerzése és feldolgozása során több forrást és eszközt is igénybe vettem:

- **Pixabay.com és YouTube:** A környezeti alapzajok és bizonyos hangeffektek forrásként szolgáltak.
- **Audacity:** Ezen nyílt forráskódú hangszerkesztő segítségével vágtam és normalizáltam a letöltött fájlokat, valamint rögzítettem és kevertem saját hangeffekteket is a karakterek interakcióihoz.

A játék vizuális világát, beleértve a sprite-okat, a felhasználói felület (UI) elemeit és egyéb technikai csomagokat, a `Unity Asset Store`-ból szereztem be. A projekt során felhasznált legfontosabb külső erőforrások és készítőik az alábbiak:

- **Shezi Khan:** 2D Adventure Backgrounds Pack [9]
- **Stets Dev:** Tower 2D Sprite Pack [4]
- **MiMU STUDIO:** Miniature Army 2D V.1 [19]
- **ArtZombie:** StoreIconUI [1]
- **Black Hammer:** Fantasy Wooden GUI : Free [5]

A vizuális elemek integrálásánál elsődleges szempont volt az egységes, középkori hangulat megőrzése és a WebGL platformon is hatékony, alacsony memóriahasználatú megjelenítés. A szöveges információk olvashatóságát minden felbontás mellett a `TextMeshPro` technológia garantálja.

8. fejezet

Szerveroldali megvalósítás és integráció

Az EduTower ökoszisztéma egyik legkritikusabb eleme a Node.js alapon nyugvó köztesréteg, amely hídként funkcionál a tanulásmenedzsment rendszer (Moodle) és a kliensoldali játékmenet között. Ez a fejezet részletezi az alkalmazott technológiákat, a hálózati architektúrát és a pontszám-szinkronizáció technikai hátterét.

8.1. Alkalmazott technológiák és architektúra

A szerveroldali logika egy Node.js [15]környezetben futó Express [18] keretrendszerre épül. A választás azért esett erre a technológiára, mert az eseményvezérelt, nem blokkoló I/O modellje kiválóan alkalmas a WebGL kliensek és az LMS rendszerek közötti aszinkron kommunikáció kezelésére.

A jelenlegi fázisban a rendszer az alábbi infrastruktúrán üzemel:

- **Hosting:** A szerver jelenleg egy lokális munkaállomáson fut a 3000-es porton.
- **Domain:** A Namecheap szolgáltatón keresztül regisztrált `bmedutower.space` cím biztosítja a fix elérési pontot. [14]
- **Hálózati csatorna:** A lokális környezet és az internet közötti biztonságos kapcsolatot a Cloudflare Zero Trust (Tunnel) technológiája biztosítja. Ez az iparági sztenderd megoldás lehetővé teszi a 3000-es port biztonságos publikálását port-továbbítás (port-forwarding) nélkül, beépített DDoS védelem mellett. [3]

Fontos megjegyezni, hogy bár a prototípus jelenleg egy lokális gépről szolgáltat, az alkalmazott Node.js alapú architektúra és a Cloudflare alagút miatt a rendszer bármikor, minimális konfiguráció-módosítással áttelepíthető egy dedikált távoli szerverre vagy felhőalapú környezetbe.

8.2. A server.js felépítése és az LTI logika

A szerver központi eleme a `server.js`, amely két fő feladatot lát el: fogadja a Moodle-ből érkező indítási kéréseket (LTI Launch) [7], valamint kezeli a játék végén beküldött eredményeket.

```
const express = require('express');
const bodyParser = require('body-parser');
const crypto = require('crypto');
const app = express();

const sessions = new Map();

// Moodle LTI indítási végpont
app.post('/lti', (req, res) => {
  const userId = req.body.user_id || 'Ismeretlen_ID';
  const level = req.query.level || '1';
  const outcomeUrl = req.body.lis_outcome_service_url;
  const sourcedId = req.body.lis_result_sourcedid;
  const token = crypto.randomBytes(16).toString('hex');

  // Munkamenet tarolasa a pontszam visszakuldeshez
  if (outcomeUrl && sourcedId) {
    sessions.set(token, {
      outcomeUrl: outcomeUrl,
      sourcedId: sourcedId
    });
  }

  // Atiranyitas a Unity jatekhoz a parameterekkel
  res.redirect(`/?level=${level}&userid=${userId}&token=${token}`);
});
```

Kódrészlet 8.1. A szerveroldali LTI indítási logika

A kód érdekessége a token-alapú munkamenet-kezelés. Az LTI indításkor a szerver egy egyedi azonosítót generál, amellyel „párosítja” a Moodle által küldött `outcomeUrl`-t és a hallgatót azonosító `sourcedId`-t. Ezt követően a rendszer átirányítja a felhasználót a `public` mappában található WebGL kliensre.

8.3. Adat-visszacsatolás és pontszám-szinkronizáció

Amikor a hallgató befejezi a szintet, a Unity játék egy belső API híváson keresztül (`/api/-score`) küldi vissza az eredményt a szervernek. A szerver ezután egy OAuth 1.0a [6] aláírással hitelesített XML üzenetet generál (POX Envelope), és továbbítja azt a Moodle felé.

```

app.post('/api/score', async (req, res) => {
  const { userId, finalScore, token } = req.body;
  const sessionData = sessions.get(token);

  if (!sessionData) return res.json({ status: "error" });

  // Normalizálás 0.0 és 1.0 köze a Moodle számára
  const normalizedScore = (Math.max(0, Math.min(100, finalScore)) /
    100).toFixed(2);

  const xmlBody = `<?xml version="1.0" encoding="UTF-8"?>
<imsx_POXEnvelopeRequest xmlns="http://www.imsglobal.org/services/
ltiv1p1/xsd/imsoms_v1p0">
  <imsx_POXBody>
    <replaceResultRequest>
      <resultRecord>
        <sourcedGUID><sourcedId>${sessionData.sourcedId}</
        sourcedId></sourcedGUID>
        <result><resultScore><textString>${normalizedScore}</
        textString></resultScore></result>
      </resultRecord>
    </replaceResultRequest>
  </imsx_POXBody>
</imsx_POXEnvelopeRequest>`;

  // OAuth alairas es kuldes Axios-szal...
});

```

Kódrészlet 8.2. Pontszám-visszaküldés OAuth hitelesítéssel

A `crypto` modul segítségével generált HMAC-SHA1 aláírás garantálja, hogy a Moodle csak hiteles forrásból fogadjon el osztályzatokat. Ez a zárt láncú folyamat biztosítja az automatizált és biztonságos értékelést.

9. fejezet

Mesterséges intelligencia a játékban

A Code Siege projekt egyik leginnovatívabb, jelenleg kísérleti fázisban lévő eleme a beépített neurális háló. Az MI réteg célja nem a komplex játékmenet-vezérlés, hanem a hallgatói teljesítmény elemzése és kategorizálása a későbbi személyre szabott oktatási visszacsatolás érdekében.

9.1. Könnyű integráció és a Synaptic.js

A szerveroldali MI megvalósításához a `Synaptic.js` [2] könyvtárat választottam. A döntés elsődleges oka a rendkívül könnyű fejleszthetőség és a Node.js környezetbe való zökkenőmentes integráció volt. Mivel a könyvtár architektúra-független, lehetővé tette egy többretegű perceptron (MLP) gyors felépítését anélkül, hogy komplex matematikai keretrendszerket (például TensorFlow) kellett volna telepíteni.

```
const { Layer, Network, Trainer } = require('synaptic');

// 2 bemenet (pont, ido), 3 rejtett, 3 kimenet (kategoriak)
const inputLayer = new Layer(2);
const hiddenLayer = new Layer(3);
const outputLayer = new Layer(3);

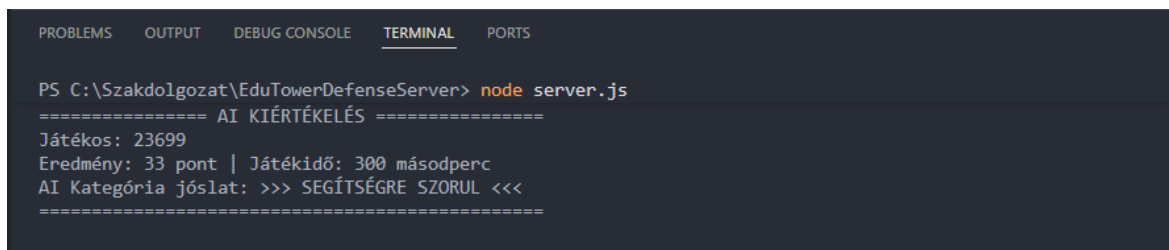
inputLayer.project(hiddenLayer);
hiddenLayer.project(outputLayer);

const aiNetwork = new Network({
  input: inputLayer,
  hidden: [hiddenLayer],
  output: outputLayer
});
```

Kódrészlet 9.1. A neurális háló gyors inicializálása és tanítása

9.2. Jelenlegi állapot és konzolos visszacsatolás

Ebben a fázisban az MI egyfajta „néma megfigyelőként” üzemel. A játék végén a szerver megkapja a hallgató pontszámát és a játékidőt, majd a hálózat aktiválása után a konzolban naplózza a jósolt kategóriát (Profi, Átlagos, Segítségre szorul).



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Szakdolgozat\EduTowerDefenseServer> node server.js
===== AI KIÉRTÉKELÉS =====
Játékos: 23699
Eredmény: 33 pont | Játékidő: 300 másodperc
AI Kategória jósolat: >>> SEGÍTSÉGRE SZORUL <<<
=====
```

9.1. ábra. Az AI kiértékelés eredménye a szerveroldali konzolon

9.3. A jövőbeli fejlődés iránya

Bár a jelenlegi megvalósítás vizuális visszacsatolást még nem nyújt a hallgatónak, a technológiai alapok készen állnak a továbblépésre. A 11. fejezetben (Bővítési tervek) részletesen kifejttem, hogyan fogja a rendszer a jövőben egy saját felhasználói felületen (UI) keresztül, részletes és látványos predikciókkal segíteni a hallgatók tanulási folyamatát.

10. fejezet

Felhasználói élmény és dokumentáció

Ez a fejezet a Code Siege projekt felhasználói oldalát mutatja be, részletezve a hallgatói élményt a bejelentkezéstől kezdve a pontszámok megtekintéséig. A rendszer célja a zökkenőmentes integráció, amely nem töri meg a tanulási folyamatot, hanem szervesen illeszkedik az egyetemi oktatási környezetbe.

10.1. Célcsoport és alkalmazhatóság

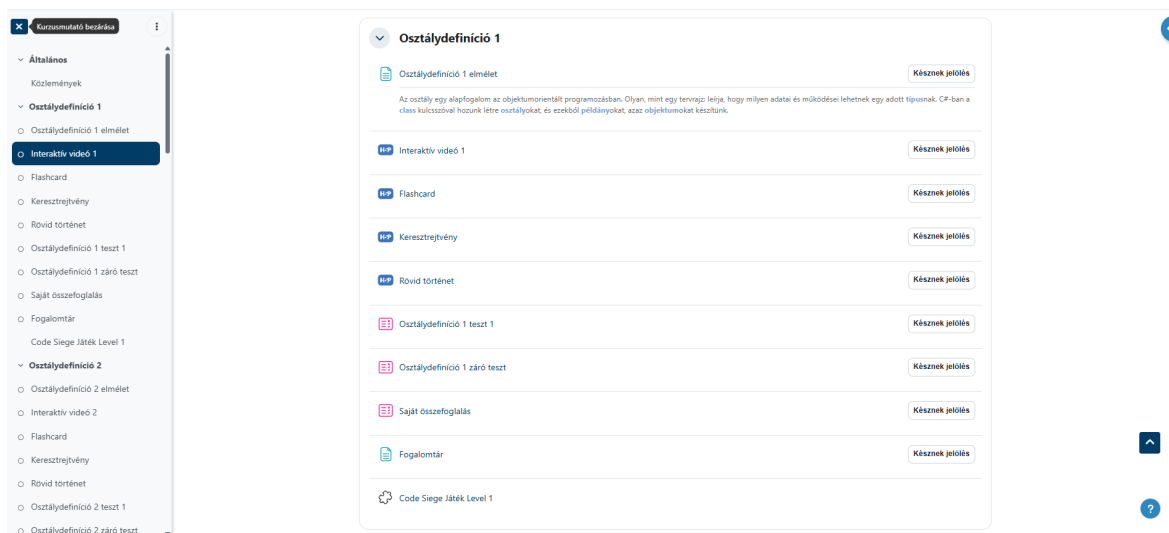
A szoftver elsődleges célcsoportját az informatika területén tanuló egyetemi hallgatók alkotják, akik az objektumorientált programozás alapjaival ismerkednek. Fontos azonban megjegyezni, hogy a keretrendszer moduláris felépítése miatt a megoldás könnyen adaptálható más karok (például műszaki vagy gazdasági szakok) specifikus igényeihez is. Emellett a játékosított környezet és az azonnali visszacsatolás módszertana a középiskolai oktatásban is hatékonyan alkalmazható, segítve a fiatalabb generációk algoritmikus gondolkodásának fejlesztését.

10.2. Az interaktív tanulási út

A felhasználói élmény a hallgató számára már jól ismert Moodle felületen kezdődik. A folyamat lépései a következők:

1. **Belépés és elméleti alapozás:** A hallgató a kurzusba belépve egy rövid, lényegre törő elméleti összefoglalót talál. A szövegben a kulcsfontosságú szakkifejezések aktív linkként jelennek meg, amelyekre rákattintva a hallgató azonnal láthatja a pontos definíciót a beépített fogalomtár segítségével.
2. **Interaktív feladatok:** Az elméletet különböző interaktív feladatok követik (például feleletválasztós tesztek, kódkiegészítések), amelyek segítenek fenntartani a hallgató érdeklődését és elmélyíteni a tudást.

3. **A játék mint tananyag elem:** A feladatsor részeként jelenik meg maga a Code Siege játék is. A technikai integrációnak köszönhetően a hallgató a Moodle keretrendszerből való kilépés nélkül, közvetlenül a böngészőben játszhat, megőrizve a tanulás folytonosságát.



10.1. ábra. A tananyag felépítése és az interaktív elemek megjelenése a Moodle felületén

10.3. Integrált értékelés és visszacsatolás

A játékelmény szerves része a jutalmazás. Minden szint végén a tanuló pontos visszajelzést kap az elért arany (Gold) mennyiségéről és a megszerzett pontszámáról. Ez az azonnali sikerélmény növeli a motivációt a következő modulok elvégzésére.

A projekt kiemelkedő tulajdonsága a pontszámok központosított kezelése. Amennyiben a hallgató később szeretné megtekinteni az előrehaladását, nem kell visszalépnie a játékba: az elért eredmények a Moodle *Gradebook* (osztályozónapló) felületén jelennek meg, az összes többi teszt és feladat eredménye mellett.

A tanárok szintén ezen a központi felületen látják az összes hallgató eredményét. A Moodle automatikusan összegzi az adatokat, így a játék zökkenőmentesen illeszkedik a féléves követelményrendszerbe, megkönnyítve az oktatók számára az értékelési folyamat adminisztrációját.

Értékelési tétel	Számított súly	Értékelés	Tartomány	Százalék	Visszajelzés	Hozzájárulás a kurzus összpontszámához
▼ Objektum-orientált programozás – C# gyakorlat						
TESZT Osztálydefiníció 1 teszt 1	33,33 %	✗ 22,50 ...	0–100	22,50 %		7,50 %
TESZT Osztálydefiníció 1 záró teszt	0,00 % (Üres)	- ...	0–100	-		0,00 %
TESZT Osztálydefiníció 2 teszt 1	0,00 % (Üres)	- ...	0–100	-		0,00 %

10.2. ábra. A Moodle Gradebook felülete

KÜLSŐ ESZKÖZ Code Siege Level 2	33,33 %	33,00 ...	0–100	33,00 %		11,00 %
KÜLSŐ ESZKÖZ Code Siege Level 3	33,33 %	33,00 ...	0–100	33,00 %		11,00 %
Σ ÖSSZEGZÉS Kurzus összegezve	-	88,50	0–300	29,50 %		-

10.3. ábra. A játékban elért eredmények megjelenése a Moodle Gradebook felületén

11. fejezet

Bővítési tervek és jövőkép

A Code Siege jelenlegi formájában egy működő prototípus, amely sikeresen integrálja a játékalapú tanulást és az automatizált értékelést. A fejlesztés során azonban számos olyan irányvonalat azonosítottam, amelyek a jövőben tovább emelhetik a rendszer oktatási értékét és technikai színvonalát.

11.1. Komplex, MI-alapú kiértékelő rendszer

A projekt legfontosabb tervezett fejlesztése a neurális háló alapú visszacsatolás kiterjesztése. A cél egy olyan komplex diagnosztikai rendszer létrehozása, amely a jelenlegi egyszavas kategóriák helyett egy dedikált felhasználói felületen (UI) keresztül nyújt részletes elemzést.

A hálózatot úgy fejleszteném tovább, hogy az képes legyen témakörökre lebontva értékelni a hallgatót. A rendszer egész mondatos, természetes nyelvi visszajelzést adna, például: „Az osztályok és objektumok kezelése kiváló, azonban az öröklődés koncepciója még bizonytalan.” Az MI javaslatokat is tenne, megjelölve a tananyag azon konkrét részeit, amelyeket a hallgatónak érdemes lenne újra átnéznie a fejlődés érdekében.

11.2. Infrastrukturális modernizáció

A jelenlegi megvalósítás – bár iparági sztenderd biztonsági megoldásokat használ – egy lokális munkaállomáson üzemel. A professzionális felhasználás és a nagyobb hallgatói létszám kiszolgálása érdekében a következő lépés a teljes rendszer áthelyezése egy dedikált felhőalapú szerverre (például AWS, Azure vagy DigitalOcean).

A valós szerverre való átállás nemcsak a rendelkezésre állást javítaná, hanem lehetővé tenné a Node.js szerver skálázását, a terheléelosztást, valamint egy robusztusabb adatbázis-kezelő (például PostgreSQL vagy MongoDB) integrálását is a hallgatói adatok hosszú távú tárolásához.

11.3. Tartalmi bővítés: szintek és karakterosztályok

Az új szintek integrálását a karbantartás és a fejlődés kritikus részének tartom. A jövőbeli tanulói visszajelzések alapján tervezem a feladatsorok bővítését, amit a jelenlegi Scriptable-Object alapú adatkezelési rendszernek köszönhetően a forráskód módosítása nélkül, hatékonyan meg tudok tenni.

A stratégiai mélységet új egységtípusok (például varázsló vagy támogató egységek) bevezetése adná, amelyek bonyolultabb kódolási mintákat igényelnének a hallgatóktól. Ez nemcsak a játékélményt javítaná, hanem összetettebb algoritmusok megírására ösztönözné a diákokat.

11.4. Mobil kompatibilitás és platformbővítés

A modern oktatási eszközökkel szemben alapvető elvárás a hordozhatóság. A Unity motor keresztplatformos képességeit kihasználva a jövőbeli célok között szerepel egy dedikált mobil verzió elkészítése is. Ez lehetővé tenné a hallgatók számára, hogy ne csak asztali környezetben, hanem táblagépen vagy okostelefonon is elvégezhessék a feladatokat. Ehhez szükséges a felhasználói felület (UI) reszponzív átalakítása és az érintőképernyős vezérlés optimalizálása, megnyitva az utat a „tanulás bárhol, bármikor” koncepciója előtt.

11.5. Felhasználói élmény (Quality of Life) fejlesztések

A gördülékeny tanulási élmény érdekében több kényelmi funkció bevezetése is a tervek között szerepel:

- **Intelligens kódszerkesztő:** A böngészőben futó kódolási ablak bővítése szintaxis-kiemeléssel és alapvető automatikus kiegészítéssel.
- **Részletesebb tutorial rendszer:** Interaktívabb segítő szövegek és vizuális jelzések, amelyek még gyorsabbá teszik a szoftver kezelésének elsajátítását.

12. fejezet

Összegzés és konklúzió

Szakedolgozatom célja egy olyan komplex, interaktív oktatási ökoszisztéma létrehozása volt, amely a modern technológia eszközeivel teszi élvezhetőbbé és hatékonyabbá az objektum-orientált programozás oktatását. A fejlesztés eredményeként létrejött „Code Siege” játék és a köré épülő infrastrukturális híd bebizonyította, hogy a játékosítás (gamification) és az automatizált visszacsatolás életképes alternatívát nyújt a hagyományos oktatási módszerek mellett.

12.1. Az ökoszisztéma működése és a megoldott problémák

A rendszer sikeresen hidalja át a szakadékot az elméleti tananyag és a gyakorlati alkalmazás között. Az ökoszisztéma három fő oszlopa – a Moodle keretrendszer, a Node.js köztesréteg és a Unity-alapú játékmotor – szoros egységben működik. A megoldott technikai kihívások közül kiemelkedik a WebGL platform korlátainak leküzdése a kódkiértékelés során, valamint az LTI protokollon keresztüli biztonságos, emberi beavatkozást nem igénylő pontszám-visszaküldés.

A projekt elsődlegesen azt a problémát orvosolja, amellyel sok kezdő programozó szembesül: az absztrakt fogalmak vizualizációjának hiányát. A játékban a megírt kódsorok azonnali, látható hatást gyakorolnak a virtuális térre, ami segít az összefüggések mélyebb megértésében.

12.2. Létjogosultság a modern oktatásban

A szakedolgozatban bemutatott rendszer létjogosultságát az adja, hogy ott éri el a hallgatókat, ahol a leginkább otthon érzik magukat: egy interaktív, vizuális környezetben. A Moodle-be integrált megoldás nem töri meg a tanulási folyamatot, hanem szervesen illeszkedik a tantárgyi struktúrába, miközben a tanárok számára is értékes, automatizált adminisztrációt biztosít.

12.3. Szakmai fejlődés és technológiai tapasztalatok

A szakdolgozat elkészítése a hároméves egyetemi tanulmányaim szinte minden területét érintette. A fejlesztés során a korábban tanult elméleti alapok – mint a számítógépes hálózatok (Cloudflare, HTTP kommunikáció), az adatkezelés (YAML, ScriptableObjects) és az objektumorientált programozás (Manager-architektúra, interfészek) – a gyakorlatban nyertek értelmet.

A munka során számos új technológiával ismerkedtem meg mélyebben:

- **Unity:** A játékmotor mélyebb megismerése, a WebGL build folyamatok és a modularításra épülő fejlesztés terén.
- **Moodle LTI:** A Learning Tools Interoperability szabvány elsajátítása, amely kulcsfontosságú volt a külső rendszerek integrálásához.
- **H5P:** Az interaktív tananyagfejlesztés alapjainak megismerése. Bár a jelenlegi céljaimhoz elegendőnek bizonyult a Moodle beépített H5P eszköztára, tisztában vagyok a fejlettebb asztali alkalmazások, mint például a **Lumi** [10] nyújtotta szélesebb körű lehetőségekkel, amelyek a jövőbeli fejlesztések alapjául szolgálhatnak.

Összegzésként elmondható, hogy a projekt nemcsak egy szoftvertechnológiai feladat megoldása volt, hanem egy pedagógiai kísérlet is, amely rávilágított az interaktív megoldások fontosságára a modern mérnökképzésben. A rendszer alapjai stabilak, bővíthetők, és készen állnak arra, hogy a jövőben még több hallgató számára tegyék élménnyé a programozás tanulását.

13. fejezet

Summary and Conclusion

The primary objective of my thesis was to create a complex, interactive educational ecosystem that utilizes modern technological tools to make the teaching of Object-Oriented Programming (OOP) more engaging and effective. The resulting "Code Siege" game and the supporting infrastructural bridge have demonstrated that gamification and automated feedback provide a viable and potent alternative to traditional teaching methods.

13.1. The Ecosystem in Operation and Problems Solved

The system successfully bridges the gap between theoretical curriculum and practical application. The ecosystem's three main pillars—the Moodle framework, the Node.js middleware, and the Unity-based game engine—operate in close harmony. Among the technical challenges addressed, overcoming the limitations of the WebGL platform during real-time code evaluation and implementing secure, automated score reporting via the LTI protocol without requiring human intervention are particularly noteworthy.

The project primarily addresses a common hurdle faced by many novice programmers: the lack of visualization for abstract concepts. Within the game, written lines of code have an immediate, visible impact on the virtual environment, which fosters a deeper understanding of underlying programming logic and relationships.

13.2. Relevance in Modern Education

The relevance of the system presented in this thesis lies in its ability to reach students in an environment where they feel most at home: an interactive, visual setting. The solution, integrated directly into Moodle, does not disrupt the learning process; instead, it fits organically into the course structure while providing valuable automated administration for instructors.

13.3. Professional Growth and Technological Experiences

The preparation of this thesis touched upon nearly every area of my three years of university studies. During development, previously learned theoretical foundations—such as computer networks (Cloudflare, HTTP communication), data management (YAML, ScriptableObjects), and object-oriented programming (Manager architecture, interfaces)—gained practical meaning.

During the project, I gained in-depth experience with several new technologies:

- **Unity:** Deepened knowledge of the game engine, specifically regarding WebGL build processes and modularity-based development.
- **Moodle LTI:** Mastering the Learning Tools Interoperability standard, which was crucial for integrating external systems.
- **H5P:** Learning the fundamentals of interactive content development. While Moodle's built-in H5P tools proved sufficient for current goals, I am aware of the broader possibilities offered by advanced desktop applications, such as **Lumi** [10], which may serve as a basis for future enhancements.

In summary, the project was not only a software engineering task but also a pedagogical experiment that highlighted the importance of interactive solutions in modern engineering education. The foundations of the system are stable, scalable, and ready to make the process of learning programming an engaging experience for even more students in the future.

Forrásjegyzék

A linkek utoljára ellenőrizve: 2026. 05. 03.

- [1] ArtZombie. Storeiconui. <https://assetstore.unity.com/packages/2d/gui/icons/storeiconui-124152>.
- [2] Juan Cazala. Synaptic.js: Architecture-free neural network library for node.js and the browser. <https://github.com/cazala/synaptic>. Utoljára ellenőrizve: 2026. 05. 03.
- [3] Cloudflare. Cloudflare tunnel documentation. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>.
- [4] Stets Dev. Tower 2d sprite pack. <https://assetstore.unity.com/packages/2d/environments/tower-2d-sprite-pack-71771>.
- [5] Black Hammer. Fantasy wooden gui : Free. <https://assetstore.unity.com/packages/2d/gui/fantasy-wooden-gui-free-103811>.
- [6] E. Hammer-Lahav. The oauth 1.0 protocol. <https://oauth.net/core/1.0a/>.
- [7] IMS Global. Lti v1.1 implementation guide. <https://www.imsglobal.org/specs/ltiv1p1/implementation-guide>.
- [8] Joubel. H5p - create and share rich html5 content and applications. <https://h5p.org/>.
- [9] Shezi Khan. 2d adventure backgrounds pack. <https://assetstore.unity.com/packages/2d/environments/2d-adventure-backgrounds-pack-309711>.
- [10] Lumi Education. Lumi - desktop app to create, edit and share h5p content. <https://lumi.education/>.
- [11] Microsoft. Interfaces - c programming guide. <https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/interfaces/>.

- [12] Microsoft. .net regular expressions. <https://learn.microsoft.com/en-us/dotnet/standard/base-types/regular-expressions>.
- [13] Moodle HQ. Moodle documentation. https://docs.moodle.org/502/en/Main_page.
- [14] Namecheap. Namecheap - domain name search and registration. <https://www.namecheap.com/>.
- [15] Node.js Foundation. Node.js v20.x documentation. <https://nodejs.org/en/about>.
- [16] ELearning Portal. Moodle to moodle lti integration. https://youtu.be/ZyUf9oMEemSA?si=NZyLcsFA_-7oiS8z.
- [17] Sebastien Ros. Jint - javascript interpreter for .net. <https://github.com/sebastienros/jint>.
- [18] StrongLoop. Express - fast, unopinionated, minimalist web framework for node.js. <https://expressjs.com/>.
- [19] MiMU STUDIO. Miniature army 2d v.1. <https://assetstore.unity.com/packages/2d/characters/miniature-army-2d-v-1-medieval-style-72935>.
- [20] Unity Technologies. Unity manual: Coroutines. <https://docs.unity3d.com/Manual/Coroutines.html>.
- [21] Unity Technologies. Unity manual: Il2cpp overview. <https://docs.unity3d.com/Manual/IL2CPP.html>.
- [22] Unity Technologies. Unity manual: Prefabs. <https://docs.unity3d.com/Manual/Prefabs.html>.
- [23] Unity Technologies. Unity manual: Project window. <https://docs.unity3d.com/Manual/ProjectView.html>.
- [24] Unity Technologies. Unity manual: Scene management. <https://docs.unity3d.com/Manual/CreatingScenes.html>.
- [25] Unity Technologies. Unity manual: ScriptableObject. <https://docs.unity3d.com/Manual/class-ScriptableObject.html>.
- [26] Unity Technologies. Unity manual: Ui toolkit. <https://docs.unity3d.com/Manual/UIElements.html>.

- [27] Unity Technologies. Unity manual: WebGL development. <https://docs.unity3d.com/Manual/webgl-gettingstarted.html>.
- [28] Unity Technologies. Unity real-time development platform. <https://unity.com/>.

GitHub adattár használati útmutató

A szakdolgozathoz tartozó minden digitális állomány, forráskód és dokumentáció elérhető a lenti nyilvános GitHub adattárban. Ez az oldal segédletként szolgál a repozitórium tartalmának és szerkezetének áttekintéséhez.

Az adattár elérhetősége:

<https://github.com/BMartiiin/BaloghMartinSzakdolgozat>

A repozitórium mappaszerkezete és tartalma:

- **CodeSiegeForráskód:** A Unity játékmotorban fejlesztett *Code Siege* oktatójáték teljes C# forráskódját tartalmazza.
- **CodeSiegeFuttatható:** Itt található a játék WebGL platformra exportált változata.
- **KépekÉsVideók:** Tartalmazza a felhasználói élményt bemutató képeket és videókat.
- **SzakdolgozatDokumentum:** Tartalmazza a szakdolgozat végleges változatát PDF formátumban (`szakdolgozat.pdf`), valamint a dolgozat elkészítéséhez használt teljes $\text{\LaTeX}$ forráskódot és a hozzá tartozó ábrákat.
- **SzerverKód:** A `server.js` file-t tartalmazza.
- **README.md:** Az adattár gyökérkönyvtárában található.